

10 Pipeline Orchestration

Course Name: Data Management for Machine Learning

Assignment Title: End-to-End Data Management Pipeline for a Recommendation System

Assignment: Group 51 - Data management for Machine Learning Group 51

Team Members

Team Member Name	Team Member ID
BANSHIDHAR RATH	2025AE05346
JITENDRA KUMAR TIWARI	2025AE05518
KATBA ANKIT CHIMANBHAI	2025AE05229
NAVEEN SURATHU	2025AE05492

1. Objective

This report documents pipeline orchestration for the recommendation pipeline by embedding the Prefect notebook content from `src\06-orchestration\pipeline_orchestration_prefect.ipynb` and printing the latest orchestration logs from `logs\orchestration`.

2. Objective Coverage

- Automate the end-to-end pipeline using Prefect.
- Capture orchestration code for ingestion, validation, preparation, transformation, feature store, and model training.
- Print latest orchestration execution logs.
- Render long text as wrapped Paragraph cells.
- Wrap long log lines before passing them to Preformatted.
- Break long file paths only at safe separators such as `\`, `/`, `_`, `-`, and `=`.

3. Supporting Artifacts

Artifact	Relative Path	Last Modified	Size
pipeline_orchestration_prefect.ipynb	src\06-orchestration\pipeline_orchestration_prefect.ipynb	2026-04-29 19:31:52	96.35 KB
run_validation.txt	src\02-validation\run_validation.txt	2026-04-28 20:14:12	46.94 KB
compare_models_20260429_164131.log	logs\orchestration\compare_models_20260429_164131.log	2026-04-29 17:07:14	1.01 KB
train_content_based_20260429_164131.log	logs\orchestration\train_content_based_20260429_164131.log	2026-04-29 17:07:04	1.09 KB
train_svd_20260429_164131.log	logs\orchestration\train_svd_20260429_164131.log	2026-04-29 17:06:32	0.93 KB
train_item_based_20260429_164131.log	logs\orchestration\train_item_based_20260429_164131.log	2026-04-29 17:06:27	1.07 KB

Artifact	Relative Path	Last Modified	Size
registry_loader_20260429_164131.log	logs\orchestration\registry_loader_20260429_164131.log	2026-04-29 17:05:35	1.04 KB

4. Orchestration Summary

orchestration_tool	notebook_found	notebook_path	log_folder_found	latest_logs_found
Prefect	True	src\06-orchestration\pipeline_orchestration_prefect.ipynb	True	5

5. Latest Orchestration Logs Summary

Log File	Relative Path	Last Modified	Size
compare_models_20260429_164131.log	logs\orchestration\compare_models_20260429_164131.log	2026-04-29 17:07:14	1.01 KB
train_content_based_20260429_164131.log	logs\orchestration\train_content_based_20260429_164131.log	2026-04-29 17:07:04	1.09 KB
train_svd_20260429_164131.log	logs\orchestration\train_svd_20260429_164131.log	2026-04-29 17:06:32	0.93 KB
train_item_based_20260429_164131.log	logs\orchestration\train_item_based_20260429_164131.log	2026-04-29 17:06:27	1.07 KB
registry_loader_20260429_164131.log	logs\orchestration\registry_loader_20260429_164131.log	2026-04-29 17:05:35	1.04 KB

6. Orchestration DAG / Notebook Content

Path: src\06-orchestration\pipeline_orchestration_prefect.ipynb

```
# ---- Code Cell 1 ----
# Cell 1 – Setup paths and folders
from pathlib import Path
import os
import re
import json
from datetime import datetime

PROJECT_ROOT = Path(r"C:\Users\barath\recomart-pipeline")
RUN_TS = datetime.now().strftime("%Y%m%d_%H%M%S")

LOG_DIR = PROJECT_ROOT / "logs" / "orchestration"
MONITOR_DIR = PROJECT_ROOT / "logs" / "monitoring"
RUN_DIR = PROJECT_ROOT / "runs" / "prefect" / RUN_TS
EXEC_NOTEBOOK_DIR = RUN_DIR / "executed_notebooks"

for p in [LOG_DIR, MONITOR_DIR, RUN_DIR, EXEC_NOTEBOOK_DIR]:
    p.mkdir(parents=True, exist_ok=True)

os.chdir(PROJECT_ROOT)

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RUN_TS:", RUN_TS)
print("Working directory:", Path.cwd())

# ---- Code Cell 2 ----
# Cell 2 – Install Prefect and dependencies

import sys
import subprocess
import importlib
import os

def ensure_package(import_name, pip_name=None):
    pip_name = pip_name or import_name
    try:
        importlib.import_module(import_name)
        print(f"[OK] {pip_name} already available")
    except ImportError:
        print(f"[INSTALL] {pip_name}")
        subprocess.check_call([sys.executable, "-m", "pip", "install", pip_name])

packages = [
    ("prefect", "prefect"),
    ("papermill", "papermill"),
    ("nbformat", "nbformat"),
    ("nbclient", "nbclient"),
    ("ipykernel", "ipykernel"),
    ("requests", "requests"),
]

for import_name, pip_name in packages:
    ensure_package(import_name, pip_name)

os.environ["PREFECT_LOGGING_LEVEL"] = "INFO"
os.environ["PYTHONIOENCODING"] = "utf-8"

print("Dependency check complete.")

# ---- Code Cell 3 ----
# Cell 3 – Start local Prefect server/UI

import socket
import time
import os
import subprocess
import sys
import webbrowser
import requests

PREFECT_UI_URL = "http://127.0.0.1:4200"
PREFECT_API_URL = "http://127.0.0.1:4200/api"
PREFECT_SERVER_LOG = LOG_DIR / f"prefect_server_{RUN_TS}.log"

def is_port_open(host="127.0.0.1", port=4200):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(1)
        return sock.connect_ex((host, port)) == 0

def wait_for_prefect_api(api_url=PREFECT_API_URL, timeout=120):
    start = time.time()
    health_url = f"{api_url}/health"
    while time.time() - start < timeout:
        try:
```

```

        response = requests.get(health_url, timeout=3)
        if response.status_code == 200:
            return True
        except Exception:
            pass
        time.sleep(2)
    return False

server_process = None

if is_port_open():
    print("Prefect server already running on port 4200.")
else:
    log_file = open(PREFECT_SERVER_LOG, "w", encoding="utf-8")
    server_process = subprocess.Popen(
        [sys.executable, "-m", "prefect", "server", "start"],
        cwd=str(PROJECT_ROOT),
        stdout=log_file,
        stderr=subprocess.STDOUT,
        env={**os.environ, "PREFECT_API_URL": PREFECT_API_URL}
    )
    print("Started Prefect server process:", server_process.pid)
    print("Server log:", PREFECT_SERVER_LOG)

ready = wait_for_prefect_api()
if not ready:
    raise RuntimeError("Prefect server did not become ready within timeout.")

os.environ["PREFECT_API_URL"] = PREFECT_API_URL
print("Prefect UI:", PREFECT_UI_URL)
print("Prefect API:", PREFECT_API_URL)

try:
    webbrowser.open(PREFECT_UI_URL)
except Exception:
    pass

# ---- Code Cell 4 ----
# Cell 4 – Define your notebook paths
NOTEBOOK_PATHS = {
    "ingest_dummyjson": PROJECT_ROOT / "src" / "01-ingestion" / "dummyjson_ingestion.ipynb",
    "ingest_retailrocket": PROJECT_ROOT / "src" / "01-ingestion" /
    "retailrocket_ingestion.ipynb",
    "validate_raw": PROJECT_ROOT / "src" / "02-validation" / "run_validation.ipynb",
    "eda_prep": PROJECT_ROOT / "src" / "03-eda-prep" / "eda_prep-copy.ipynb",
    "materialize_features": PROJECT_ROOT / "src" / "04-features" /
    "materialize_features.ipynb",
    "registry_loader": PROJECT_ROOT / "src" / "04-features" / "registry_loader.ipynb",
    "train_item_based": PROJECT_ROOT / "src" / "05-training" / "item-based-
    collaborative.ipynb",
    "train_svd": PROJECT_ROOT / "src" / "05-training" / "svd.ipynb",
    "train_content_based": PROJECT_ROOT / "src" / "05-training" / "content-based-
    training.ipynb",
    "compare_models": PROJECT_ROOT / "src" / "05-training" / "comparision.ipynb",
}

for name, path in NOTEBOOK_PATHS.items():
    print(f"{name:22} -> {'FOUND' if path.exists() else 'MISSING'} | {path}")

# ---- Code Cell 5 ----
# Cell 5 – Logging and notebook execution helpers
import logging

def _safe_name(text: str) -> str:
    return re.sub(r"^[A-Za-z0-9]+", "_", text).strip("_").lower()

def get_logger(name: str):
    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)

    if not logger.handlers:
        log_file = LOG_DIR / f"{_safe_name(name)}_{RUN_TS}.log"
        fh = logging.FileHandler(log_file, encoding="utf-8")
        sh = logging.StreamHandler(sys.stdout)

        formatter = logging.Formatter("%(asctime)s | %(levelname)s | %(name)s | %(message)s")
        fh.setFormatter(formatter)
        sh.setFormatter(formatter)

        logger.addHandler(fh)
        logger.addHandler(sh)

    return logger

def write_status(stage: str, status: str, notebook_path: Path = None, error: str = None,
output_notebook: Path = None):
    payload = {
        "run_ts": RUN_TS,
        "stage": stage,

```

```

        "status": status,
        "notebook_path": str(notebook_path) if notebook_path else None,
        "output_notebook": str(output_notebook) if output_notebook else None,
        "error": error,
        "event_ts": datetime.now().isoformat()
    }
    out_path = MONITOR_DIR / f"{_safe_name(stage)}_{status}_{RUN_TS}.json"
    out_path.write_text(json.dumps(payload, indent=2), encoding="utf-8")
    return out_path

def execute_notebook(stage: str, notebook_path: Path):
    logger = get_logger(stage)

    if not notebook_path.exists():
        write_status(stage, "failed", notebook_path=notebook_path, error="Notebook file not
found")
        raise FileNotFoundError(f"Notebook not found: {notebook_path}")

    output_dir = EXEC_NOTEBOOK_DIR / _safe_name(stage)
    output_dir.mkdir(parents=True, exist_ok=True)
    output_notebook = output_dir / f"{notebook_path.stem}_executed.ipynb"

    cmd = [
        sys.executable,
        "-m",
        "papermill",
        str(notebook_path),
        str(output_notebook),
    ]

    logger.info("Starting stage: %s", stage)
    logger.info("Command: %s", " ".join(cmd))
    write_status(stage, "started", notebook_path=notebook_path,
output_notebook=output_notebook)

    result = subprocess.run(
        cmd,
        cwd=str(PROJECT_ROOT),
        capture_output=True,
        text=True,
        env={**os.environ, "PREFECT_API_URL": PREFECT_API_URL}
    )

    if result.stdout:
        logger.info("STDOUT:\n%s", result.stdout)

    if result.stderr:
        logger.warning("STDERR:\n%s", result.stderr)

    if result.returncode != 0:
        write_status(
            stage,
            "failed",
            notebook_path=notebook_path,
            output_notebook=output_notebook,
            error=result.stderr[-3000:] if result.stderr else f"Return code
{result.returncode}"
        )
        raise RuntimeError(f"{stage} failed with return code {result.returncode}")

    write_status(stage, "success", notebook_path=notebook_path,
output_notebook=output_notebook)
    logger.info("Completed stage: %s", stage)
    return str(output_notebook)

# ---- Code Cell 6 ----
# Cell 6 – Print DAG graph
from IPython.display import HTML, display

dag_html = """
<div style="font-family: Arial, sans-serif; line-height: 1.6;">
  <h3>RecoMart Prefect Orchestration DAG</h3>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#86BC25;color:white;border-
radius:8px;">DummyJSON Ingestion</span>
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-
radius:8px;">Validation</span>
  </div>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#43B02A;color:white;border-
radius:8px;">RetailRocket Ingestion</span>
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-
radius:8px;">Validation</span>
  </div>
</div>
<div style="margin: 12px 0;">

```

```

        <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-
radius:8px;">Validation</span>
        <span style="font-size:22px;margin:0 8px;">→</span>
        <span style="display:inline-block;padding:10px 14px;background:#1D4F91;color:white;border-
radius:8px;">EDA / Preparation</span>
        <span style="font-size:22px;margin:0 8px;">→</span>
        <span style="display:inline-block;padding:10px 14px;background:#005587;color:white;border-
radius:8px;">Materialize Features</span>
        <span style="font-size:22px;margin:0 8px;">→</span>
        <span style="display:inline-block;padding:10px 14px;background:#046E38;color:white;border-
radius:8px;">Registry Loader</span>
    </div>

    <div style="margin: 12px 0;">
        <span style="display:inline-block;padding:10px 14px;background:#007CB0;color:white;border-
radius:8px;">Item-Based Training</span>
        <span style="font-size:22px;margin:0 8px;">■</span>
        <span style="display:inline-block;padding:10px 14px;background:#62B5E5;color:white;border-
radius:8px;">Content-Based Training</span>
    </div>

    <div style="margin: 12px 0;">
        <span style="display:inline-block;padding:10px 14px;background:#00A9E0;color:white;border-
radius:8px;">SVD Training</span>
        <span style="font-size:22px;margin:0 8px;">■</span>
        <span style="display:inline-block;padding:10px 14px;background:#62B5E5;color:white;border-
radius:8px;">Content-Based Training</span>
        <span style="font-size:22px;margin:0 8px;">→</span>
        <span style="display:inline-block;padding:10px 14px;background:#63666A;color:white;border-
radius:8px;">Comparison</span>
    </div>
</div>
"""

```

```
display(HTML(dag_html))
```

```
# ---- Code Cell 7 ----
```

```
# Cell 7 – Define Prefect tasks and flow
```

```
from prefect import flow, task, get_run_logger
```

```
@task(name="ingest_dummyjson", retries=2, retry_delay_seconds=20)
```

```
def ingest_dummyjson():
```

```
    return execute_notebook("ingest_dummyjson", NOTEBOOK_PATHS["ingest_dummyjson"])
```

```
@task(name="ingest_retailrocket", retries=2, retry_delay_seconds=20)
```

```
def ingest_retailrocket():
```

```
    return execute_notebook("ingest_retailrocket", NOTEBOOK_PATHS["ingest_retailrocket"])
```

```
@task(name="validate_raw", retries=1, retry_delay_seconds=10)
```

```
def validate_raw():
```

```
    return execute_notebook("validate_raw", NOTEBOOK_PATHS["validate_raw"])
```

```
@task(name="eda_prep", retries=1, retry_delay_seconds=10)
```

```
def eda_prep():
```

```
    return execute_notebook("eda_prep", NOTEBOOK_PATHS["eda_prep"])
```

```
@task(name="materialize_features", retries=1, retry_delay_seconds=10)
```

```
def materialize_features():
```

```
    return execute_notebook("materialize_features", NOTEBOOK_PATHS["materialize_features"])
```

```
@task(name="registry_loader", retries=1, retry_delay_seconds=10)
```

```
def registry_loader():
```

```
    return execute_notebook("registry_loader", NOTEBOOK_PATHS["registry_loader"])
```

```
@task(name="train_item_based", retries=0)
```

```
def train_item_based():
```

```
    return execute_notebook("train_item_based", NOTEBOOK_PATHS["train_item_based"])
```

```
@task(name="train_svd", retries=0)
```

```
def train_svd():
```

```
    return execute_notebook("train_svd", NOTEBOOK_PATHS["train_svd"])
```

```
@task(name="train_content_based", retries=0)
```

```
def train_content_based():
```

```
    return execute_notebook("train_content_based", NOTEBOOK_PATHS["train_content_based"])
```

```
@task(name="compare_models", retries=0)
```

```
def compare_models():
```

```
    return execute_notebook("compare_models", NOTEBOOK_PATHS["compare_models"])
```

```
@flow(name="recomart_assignment_prefect_flow", log_prints=True)
```

```
def recomart_assignment_prefect_flow():
```

```
    logger = get_run_logger()
```

```
    logger.info("Pipeline started")
```

```
    pipeline_status = {
```

```
        "run_ts": RUN_TS,
```

```
        "status": "started",
```

```
        "event_ts": datetime.now().isoformat()
```

```

}
(MONITOR_DIR / f"pipeline_started_{RUN_TS}.json").write_text(
    json.dumps(pipeline_status, indent=2), encoding="utf-8"
)

try:
    dummy_future = ingest_dummyjson.submit()
    retail_future = ingest_retailrocket.submit()

    dummy_future.result()
    retail_future.result()

    validate_raw.submit().result()
    eda_prep.submit().result()

    materialize_features.submit().result()
    registry_loader.submit().result()

    item_future = train_item_based.submit()
    svd_future = train_svd.submit()

    item_future.result()
    svd_future.result()

    train_content_based.submit().result()
    compare_models.submit().result()

    pipeline_status["status"] = "success"
    pipeline_status["event_ts"] = datetime.now().isoformat()
    (MONITOR_DIR / f"pipeline_success_{RUN_TS}.json").write_text(
        json.dumps(pipeline_status, indent=2), encoding="utf-8"
    )

    logger.info("Pipeline completed successfully")

except Exception as exc:
    pipeline_status["status"] = "failed"
    pipeline_status["error"] = str(exc)
    pipeline_status["event_ts"] = datetime.now().isoformat()
    (MONITOR_DIR / f"pipeline_failed_{RUN_TS}.json").write_text(
        json.dumps(pipeline_status, indent=2), encoding="utf-8"
    )
    logger.exception("Pipeline failed: %s", exc)
    raise

# ---- Code Cell 8 ----
# Cell 8 – Validate all notebook files exist
missing = [str(path) for path in NOTEBOOK_PATHS.values() if not path.exists()]

if missing:
    print("Missing notebook files:")
    for m in missing:
        print("-", m)
    raise FileNotFoundError("Fix missing notebook paths before running the flow.")
else:
    print("All notebook paths found.")

# ---- Code Cell 9 ----
# Cell 9 – Run the full orchestration flow
recomart_assignment_prefect_flow()

# ---- Code Cell 10 ----
# Cell 10 – Show logs, monitoring, and executed notebooks
print("Monitoring files:")
for p in sorted(MONITOR_DIR.glob(f"*{RUN_TS}.json")):
    print("-", p)

print("\nExecuted notebooks:")
for p in sorted(EXEC_NOTEBOOK_DIR.rglob("*.ipynb")):
    print("-", p)

print("\nTask logs:")
for p in sorted(LOG_DIR.glob(f"*{RUN_TS}.log")):
    print("-", p)

# ---- Code Cell 11 ----
# Cell 11 – Optional cleanup: stop Prefect server started by notebook
if "server_process" in globals() and server_process is not None:
    server_process.terminate()
    print(f"Stopped Prefect server process: {server_process.pid}")
else:
    print("No notebook-started Prefect server process found.")

```

7. Latest Orchestration Logs

Log File: compare_models_20260429_164131.log

Path: logs\orchestration\compare_models_20260429_164131.log

```
2026-04-29 17:07:04,258 | INFO | compare_models | Starting stage: compare_models
2026-04-29 17:07:04,262 | INFO | compare_models | Command:
c:\Users\barath\AppData\Local\anaconda3\envs\dm4ml\python.exe -m papermill
C:\Users\barath\recomart-pipeline\src\05-training\comparision.ipynb C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\compare_models\comparision_executed.ipynb
2026-04-29 17:07:14,856 | WARNING | compare_models | STDERR:
Input Notebook: C:\Users\barath\recomart-pipeline\src\05-training\comparision.ipynb
Output Notebook: C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\compare_models\comparision_executed.ipynb

Executing: 0%|          | 0/1 [00:00<?, ?cell/s]Executing notebook with kernel: dm4ml

Executing: 100%|██████████| 1/1 [00:08<00:00, 8.11s/cell]
Executing: 100%|██████████| 1/1 [00:08<00:00, 8.67s/cell]

2026-04-29 17:07:14,863 | INFO | compare_models | Completed stage: compare_models
```

Log File: train_content_based_20260429_164131.log

Path: logs\orchestration\train_content_based_20260429_164131.log

```
2026-04-29 17:06:32,306 | INFO | train_content_based | Starting stage: train_content_based
2026-04-29 17:06:32,310 | INFO | train_content_based | Command:
c:\Users\barath\AppData\Local\anaconda3\envs\dm4ml\python.exe -m papermill
C:\Users\barath\recomart-pipeline\src\05-training\content-based-training.ipynb
C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\train_content_based\content-based-training_executed.ipynb
2026-04-29 17:07:04,208 | WARNING | train_content_based | STDERR:
Input Notebook: C:\Users\barath\recomart-pipeline\src\05-training\content-based-training.ipynb
Output Notebook: C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\train_content_based\content-based-training_executed.ipynb

Executing: 0%|          | 0/1 [00:00<?, ?cell/s]Executing notebook with kernel: dm4ml

Executing: 100%|██████████| 1/1 [00:28<00:00, 28.39s/cell]
Executing: 100%|██████████| 1/1 [00:29<00:00, 29.38s/cell]

2026-04-29 17:07:04,216 | INFO | train_content_based | Completed stage: train_content_based
```

Log File: train_svd_20260429_164131.log

Path: logs\orchestration\train_svd_20260429_164131.log

```
2026-04-29 17:05:35,453 | INFO | train_svd | Starting stage: train_svd
2026-04-29 17:05:35,469 | INFO | train_svd | Command:
c:\Users\barath\AppData\Local\anaconda3\envs\dm4ml\python.exe -m papermill
C:\Users\barath\recomart-pipeline\src\05-training\svd.ipynb C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\train_svd\svd_executed.ipynb
2026-04-29 17:06:32,270 | WARNING | train_svd | STDERR:
Input Notebook: C:\Users\barath\recomart-pipeline\src\05-training\svd.ipynb
Output Notebook: C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\train_svd\svd_executed.ipynb

Executing: 0%|          | 0/2 [00:00<?, ?cell/s]Executing notebook with kernel: dm4ml

Executing: 50%|██████    | 1/2 [00:53<00:53, 53.73s/cell]
Executing: 100%|██████████| 2/2 [00:54<00:00, 27.26s/cell]

2026-04-29 17:06:32,276 | INFO | train_svd | Completed stage: train_svd
```

Log File: train_item_based_20260429_164131.log

Path: logs\orchestration\train_item_based_20260429_164131.log

```
2026-04-29 17:05:35,456 | INFO | train_item_based | Starting stage: train_item_based
2026-04-29 17:05:35,465 | INFO | train_item_based | Command:
c:\Users\barath\AppData\Local\anaconda3\envs\dm4ml\python.exe -m papermill
C:\Users\barath\recomart-pipeline\src\05-training\item-based-collaborative.ipynb
C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\train_item_based\item-based-collaborative_executed.ipynb
2026-04-29 17:06:27,047 | WARNING | train_item_based | STDERR:
```



```

Input Notebook: C:\Users\barath\recomart-pipeline\src\05-training\item-based-
collaborative.ipynb
Output Notebook: C:\Users\barath\recomart-
pipeline\runs\prefect\20260429_164131\executed_notebooks\train_item_based\item-based-
collaborative_executed.ipynb

Executing: 0%|          | 0/2 [00:00<?, ?cell/s]Executing notebook with kernel: dm4ml

Executing: 50%|██████    | 1/2 [00:48<00:48, 48.23s/cell]
Executing: 100%|██████████| 2/2 [00:49<00:00, 24.66s/cell]

2026-04-29 17:06:27,053 | INFO | train_item_based | Completed stage: train_item_based

```

Log File: registry_loader_20260429_164131.log

Path: logs\orchestration\registry_loader_20260429_164131.log

```

2026-04-29 17:05:20,452 | INFO | registry_loader | Starting stage: registry_loader
2026-04-29 17:05:20,455 | INFO | registry_loader | Command:
c:\Users\barath\AppData\Local\anaconda3\envs\dm4ml\python.exe -m papermill
C:\Users\barath\recomart-pipeline\src\04-features\registry_loader.ipynb
C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks\registry_load
er\registry_loader_executed.ipynb
2026-04-29 17:05:35,411 | WARNING | registry_loader | STDERR:
Input Notebook: C:\Users\barath\recomart-pipeline\src\04-features\registry_loader.ipynb
Output Notebook: C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebo
oks\registry_loader\registry_loader_executed.ipynb

Executing: 0%|          | 0/1 [00:00<?, ?cell/s]Executing notebook with kernel: dm4ml

Executing: 100%|██████████| 1/1 [00:08<00:00, 8.27s/cell]
Executing: 100%|██████████| 1/1 [00:08<00:00, 8.63s/cell]

2026-04-29 17:05:35,419 | INFO | registry_loader | Completed stage: registry_loader

```

8. Additional Pipeline Evidence

The validation workflow output is included as supporting pipeline evidence when available.

```

{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 2,
      "id": "fef32579-f890-4b53-8d24-2fcae64a4914",
      "metadata": {},
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [
            "PROJECT_ROOT: C:\\Users\\barath\\recomart-pipeline\\n",
            "RAW_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\n",
            "BRONZE_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\\n",
            "\\n",
            "Discovered files:\\n",
            "- events_csv: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour=14\\events.csv\\n",
            "- category_tree_csv: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-26\\load_hour=19\\category_tree.csv\\n",
            "- item_properties_part1_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part1.csv\\n",
            "- item_properties_part2_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part2.csv\\n",
            "- products_raw_json: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\products_raw.json\\n",
            "- categories_raw_json: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\categories_raw.json\\n",
            "- products_parquet: C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\products.parquet\\n",
            "- categories_parquet: C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\categories.parquet\\n",
            "\\n",
            "Initial status: FAIL\\n",
            "Final status: PASS\\n",
            "Initial issues file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_initial_20260428T144031Z.csv\\n",
            "Initial summary file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_initial_20260428T144031Z.csv\\n",
            "Revalidation issues file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_revalidated_20260428T144031Z.csv\\n",
            "Revalidation summary file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_revalidated_20260428T144031Z.csv\\n",
            "Fix log file: C:\\Users\\barath\\recomart-

```

```

pipeline\\reports\\validation\\fix_log_20260428T144031Z.csv\\n",
    "JSON report file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.json\\n",
    "PDF report file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.pdf\\n"
]
}
],
"source": [
    "# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report\\n",
    "\\n",
    "import json\\n",
    "from pathlib import Path\\n",
    "from datetime import datetime, timezone\\n",
    "\\n",
    "import pandas as pd\\n",
    "\\n",
    "from reportlab.lib import colors\\n",
    "from reportlab.lib.pagesizes import A4\\n",
    "from reportlab.lib.styles import getSampleStyleSheet\\n",
    "from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
PageBreak\\n",
    "\\n",
    "# =====\\n",
    "# RecoMart Validation - robust file discovery + validation\\n",
    "# with remediation, revalidation, and PDF reporting\\n",
    "# =====\\n",
    "\\n",
    "PROJECT_ROOT = Path.cwd().resolve()\\n",
    "while not (PROJECT_ROOT / \\src\\).exists() and PROJECT_ROOT != PROJECT_ROOT.parent:\\n",
    "    PROJECT_ROOT = PROJECT_ROOT.parent\\n",
    "\\n",
    "RAW_ROOT = PROJECT_ROOT / \\data\\ / \\raw\\\"\\n",
    "BRONZE_ROOT = PROJECT_ROOT / \\data\\ / \\bronze\\\"\\n",
    "\\n",
    "REMEDIATED_RAW_ROOT = PROJECT_ROOT / \\data\\ / \\remediated\\ / \\raw\\\"\\n",
    "REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / \\data\\ / \\remediated\\ / \\bronze\\\"\\n",
    "\\n",
    "REPORT_ROOT = PROJECT_ROOT / \\reports\\ / \\validation\\\"\\n",
    "LOG_ROOT = PROJECT_ROOT / \\logs\\\"\\n",
    "\\n",
    "for folder in [REPORT_ROOT, LOG_ROOT, REMEDIATED_RAW_ROOT, REMEDIATED_BRONZE_ROOT]:\\n",
    "    folder.mkdir(parents=True, exist_ok=True)\\n",
    "\\n",
    "UTC_NOW = datetime.now(timezone.utc)\\n",
    "RUN_ID = UTC_NOW.strftime(\\\"%Y%m%dT%H%M%S\\\")\\n",
    "RUN_TS = UTC_NOW.isoformat()\\n",
    "\\n",
    "LOG_FILE = LOG_ROOT / f\\validation_log_{RUN_ID}.json\\\"\\n",
    "INITIAL_ISSUES_FILE = REPORT_ROOT / f\\validation_issues_initial_{RUN_ID}.csv\\\"\\n",
    "INITIAL_SUMMARY_FILE = REPORT_ROOT / f\\validation_summary_initial_{RUN_ID}.csv\\\"\\n",

```

9. Conclusion

This PDF consolidates the pipeline orchestration deliverables by embedding the specified Prefect notebook content and printing the latest orchestration execution logs in a submission-ready format.